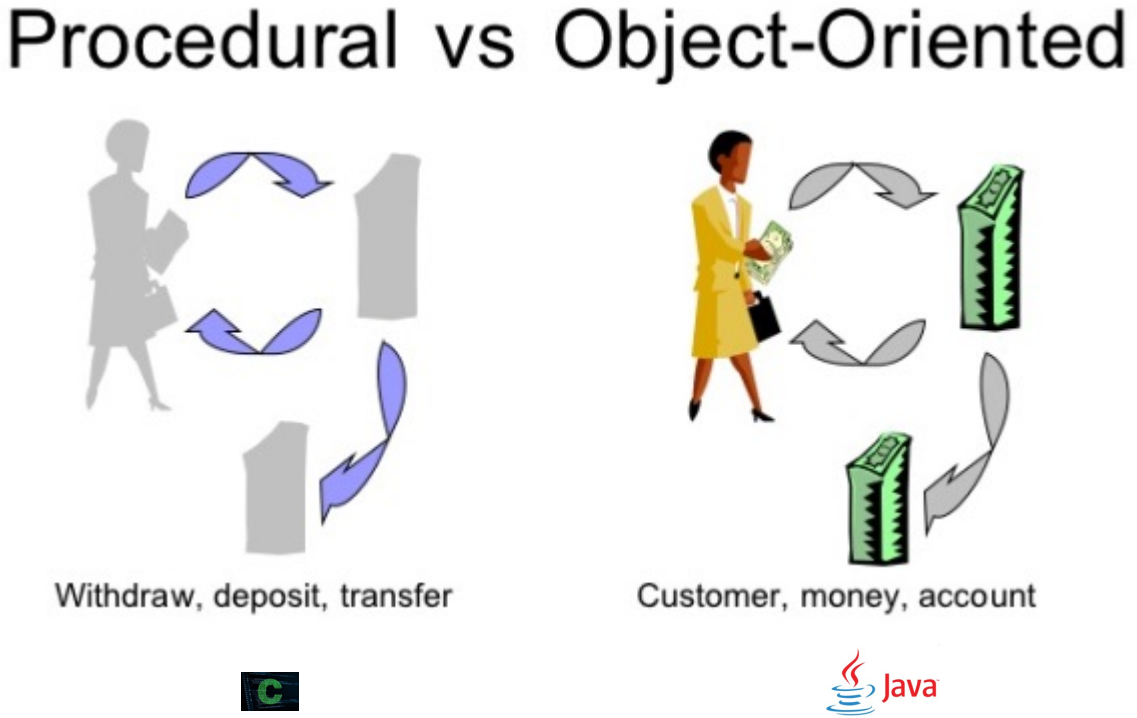
Classes and Objects

Programming Paradigms



https://www.alphansotech.com/procedural-vs-object-oriented-programming

Classes and Objects

A **class** is a template that defines the form of an object.

- It specifies both the ***code*** and ***data*** of an object.
- The ***members*** of a class are: ***fields*** and ***methods.***
  - A field is sometimes called an *instance variable*.
- A class does not exist at run-time (*).

An **object** is a value that exists at run-time.

- An object is an ***instance*** of a class.
- An object is created (allocated) with the `new` keyword.
- An object is automatically destroyed (deallocated) by garbage collection.

(*): At least for the purposes of this course…

A Prototypical Class

```
class ClassName {

     // fields
     type field1;
     type field2;
     ...

     // methods
     type method1(params...) {
          // method body
     }

     type method2(params...) {
          // method body
     }
     ...

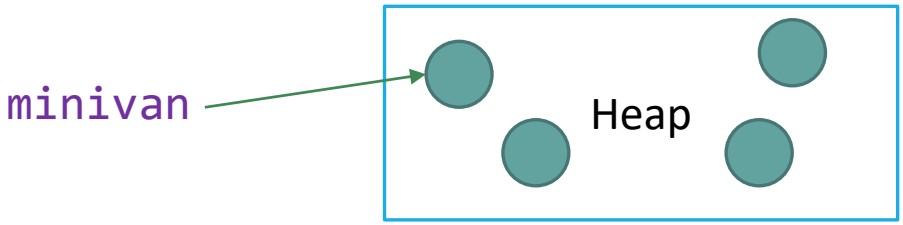
}
```

A Vehicle Class

```
class Vehicle {
    int passengers; // number of passengers.
    int fuelcap;    // fuel capacity in gallons.
    int mpg;        // fuel consumption in miles per gallon.
}
```

```
Vehicle minivan = new Vehicle();

minivan.passengers = 7;
minivan.fuelcap    = 16;
minivan.mpg        = 21;
```



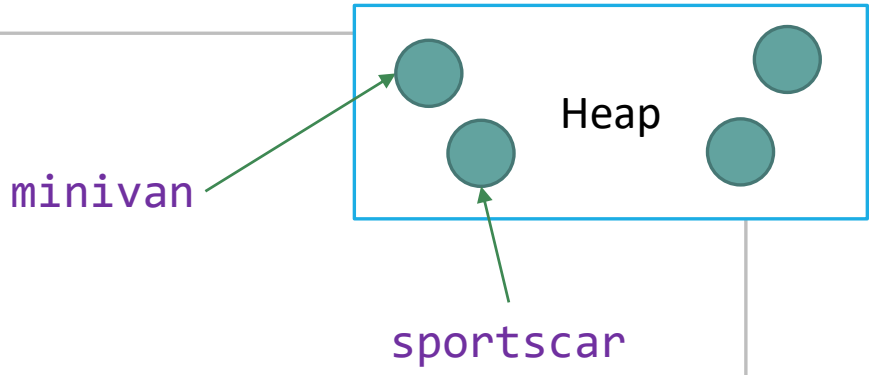
Two Vehicles

```
Vehicle minivan = new Vehicle();
minivan.passengers = 7;
minivan.fuelcap    = 16;
minivan.mpg        = 21;

Vehicle sportscar = new Vehicle();
sportscar.passengers = 2;
sportscar.fuelcap    = 14;
sportscar.mpg        = 12;

int minivanRange = minivan.fuelcap * minivan.mpg;
int sportscarRange = sportscar.fuelcap * sportscar.mpg;

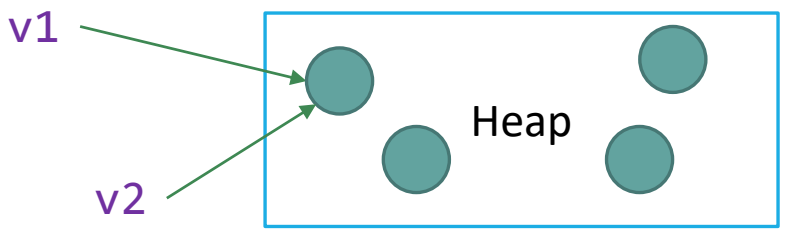
System.out.println("Range of:");
System.out.println("  minivan: " + minivanRange + " miles.");
System.out.println("  sportscar: " + sportscarRange + " miles.");
```



References and Aliasing

```
Vehicle v1 = new Vehicle();
v1.passengers = 7;
v1.fuelcap    = 16;
v1.mpg        = 21;

Vehicle v2 = v1;
v2.passengers = 0;
```



Methods

```
class Vehicle {
    int passengers; // number of passengers.
    int fuelcap;    // fuel capacity in gallons.
    int mpg;        // fuel consumption in miles per gallon.

    double fuelNeeded(int miles) {
        return (double) miles / (double) this.mpg;
    }

     double fuelNeeded2(int miles) {
        return (double) miles / (double) mpg; // equivalent.
    }
}
```

What goes in class?

A class should represent **one logical entity**.

- E.g. a vehicle, a fruit, a tennis player, a ticket, a stadium, …

A class should capture behavior related to its entity.

- E.g. a vehicle has a number of passengers,
- E.g. a fruit has a name, a taste, …,
- E.g. a ticket has a price, a purchase date, …,

A class should have as little responsibility as possible.

- E.g. a vehicle might be responsible for computing fuel consumption, but should not be responsible for computing the distance between two cities.
- If the responsibilities of a class grows to great it should ***delegate***.

Constructors

A ***constructor*** is a special method called when an object is created.

- A class comes with a **default empty constructor.**
- We can use constructors to better control object creation.
- A constructor has the same name as the class name.

```
class Vehicle {
    int passengers; // number of passengers.
    int fuelcap;    // fuel capacity in gallons.
    int mpg;        // fuel consumption in miles per gallon.

    Vehicle(int p, int f, int m) {
        this.passengers = p;
        this.fuelcap = f;
        this.mpg = m;
    }
}
```

Multiple Constructors

A class may have multiple constructors:

```
class Vehicle {
    int passengers; // number of passengers.
    int fuelcap;    // fuel capacity in gallons.
    int mpg;        // fuel consumption in miles per gallon.

    Vehicle(int f, int m) {
        this.passengers = 5;
        this.fuelcap = f;
        this.mpg = m;
    }

    Vehicle(int p, int f, int m) {
        this.passengers = p;
        this.fuelcap = f;
        this.mpg = m;
    }
}
```

Garbage Collection

We allocate objects with

```
Vehicle v1 = new Vehicle(40, 20);
Vehicle v2 = new Vehicle(60, 30);
```

but how are these objects de-allocated?

Through garbage collection!

- The JVM automatically de-allocates objects when they are no longer used.
- An object is no longer used when there are no more live references to it.